TECNOLÓGICO NACIONAL DE MÉXICO

TOLUCA



Contenido

Estudia un curso de UML 3

Entender POO 4

UML 4

Definir Clases y sus componentes 5

MyMedicalAppointments 5

Creando nuestra primera clase 5

Método Constructor 7

Static: Variables y Métodos Estáticos 8

Métodos static 8

Final: Variables y Contantes 10

Sobrecarga de Métodos y Constructores 10

Encapsulamiento: Modificadores de acceso 11

GETTERS Y SETTERS 12

Variables vs Objetos 13

Vistazo en memoria 13

Clases Anidadas 14

Clases internas y Locales a un método 15

Enumerations 17

Reutilizar código 22

¿Qué es la herencia?... DRY Don´t Repeat Yourself 22

Super y This 23

Tarea 24

Investiga la clase object y para qué sirve el método toString() 24

Polimorfismo: Sobreescritura de Métodos 25

Interfaces 26

Composición de Interfaces en Clases 26

Así se va siendo la estructura UML de nuestro proyecto… 27

Collections 28

Aplicar abstracción 34

Clases abstractas 34

Miembros Abstractos 35

Clases anónimas 36

Diferencias entre las Interfaces vs Clases Abstractas 37

Interfaces en Java 8 y 9 39

Herencia en interfaces 40

Tareas 41

Estudia excepciones 41

Estudia métodos String 41

Estructuras de árbol 41

Arrylist, map, tremap 41

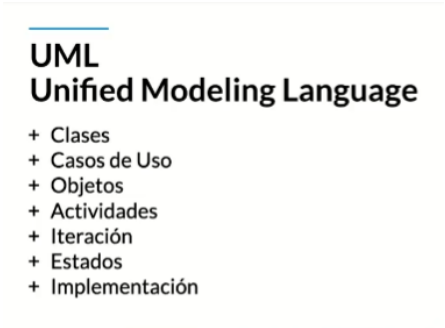
Estudia un curso de UML

Entender POO

UML



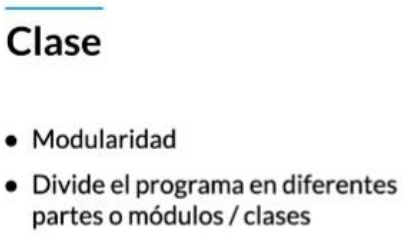
Parte de la orientación a objetos es que primero vamos a observar, analizar, graficar y programar.
UML tiene muchas aplicaciones sin embargo vamos a trabajar con esto.





A partir de que ya se tienen las clases procederemos a graficar.





Aprender a modular nuestro código será la parte esencial de POO, sin embargo, centralmente ya no se realizará con funciones, si no que será a utilizando clases.



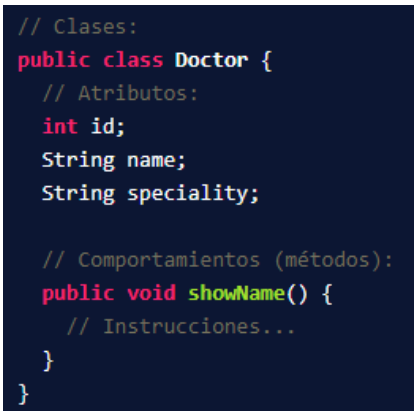
Definir Clases y sus componentes

MyMedicalAppointments

Nuestro proyecto en este curso es construir un sistema que nos permita listar y agendar nuestras citas médicas, por lo que debemos crear algunas clases para cada integrante del sistema: doctores, pacientes, entre otras.

Así vamos a crear nuestra primera clase con sus métodos y atributos:

Creando nuestra primera clase

	Primero declaramos nuestros atributos seguido de los comportamientos.
Una clase es diferente a un objeto, una clase es el molde y el objeto es el resultado a los atributos.	

Cuando utilizamos la palabra reservada new, en ese momento el objeto empieza a usar un espacio en la memoria de la computadora y es cuando se dice que se está instanciando el objeto.

Instanciando un objeto

```
myDoctor = new Doctor();
```

Nombre del Objeto — Creando el objeto

Estos dos procesos se pueden realizar en una sola línea. De esta forma ya se puede estar utilizando tanto los atributos y sus métodos.

Utilizando el objeto

```
Doctor myDoctor = new Doctor();
myDoctor.name = “Alejandro López”;
myDoctor.showName();
```

Declarando e instanciando un objeto

```
Doctor myDoctor = new Doctor();
```

Declarando el objeto — Instanciando/ Creando el objeto

```
public class Main {
    public static void main(String[] args) {

        Doctor myDoctor = new Doctor();
        myDoctor.name = "Alejandro Rodríguez";
        myDoctor.showName();

    }
}
```

Método Constructor

Método Constructor

```
myDoctor = new Doctor();
```

Método Constructor

Cuando declaramos un objeto el compilador de Java por defecto nos provee uno cuando no lo tenemos declarado, usando el nombre de la clase seguido de la sintaxis paréntesis, sin embargo, si lo podemos declarar y le podemos añadir comportamientos.

El método constructor tiene el mismo nombre de la clase sin embargo existen algunas excepciones cuando se está aplicando herencia e interfaces.

Método Constructor

- **Crea** nuevas **instancias** de una clase.
- Tiene el **mismo nombre** que la clase que inicializa.
- Usa la palabra reservada **new** para invocarlo.

Método Constructor

Usa **cero o más argumentos** contenidos dentro de los paréntesis que siguen al nombre.

No regresa un valor.

Static: Variables y Métodos Estáticos

Accesando a Métodos

```
Doctor myDoctor = new Doctor();
myDoctor.showName();

Math.random();
Math.sqrt(25);
Math.PI;
```

Existen diferentes maneras de acceder a los métodos, uno es a partir del objeto de la instancia que se creó. En el primero utilizamos nuestro objeto myDoctor para llamar a showName.

Otro es definir ese método o esa variable como estáticos y llamarlos a partir del nombre de la clase, por ejemplo, el nombre de la clase (Math) seguido del punto y el método estático o variable estática.

Métodos static

- Se puede usar en toda la clase
- Está definido por la palabra reservada **static**
- Puede ser **accesado indicando el nombre de la clase**, la notación punto y el nombre del método
- Se invoca en una clase que **no tiene instancias de la clase**.
- Los métodos o variables estáticas, no es necesario crear un objeto para entonces llamarlos.

Métodos static

```
public class Calculadora {
    public static int suma(int a, int b){
        return a+b;
    }
}
```

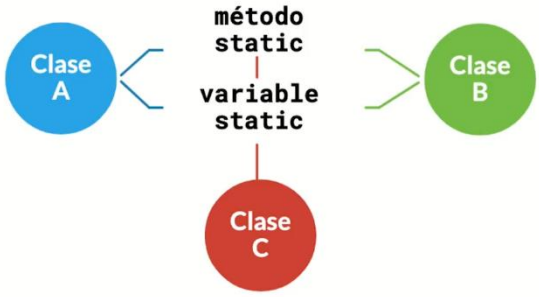
```
Calculadora.suma(5,2);
```

Miembros static

```
public class Calculadora{
    public static final double PI = 3.1415926
    public static int valor = 0;
}
```

```
Calculadora.PI;
Calculadora.valor;
```

Miembros static



Si ClaseC contiene a la variable valor, la ClaseA puede acceder a ella atraves de ClaseC.valor y 0 será el valor que se imprima, la ClaseC también puede modificar su cantidad, por ejemplo, si aumenta 1 y posteriormente la ClaseB la imprime, 1 será el valor mostrado.

Existen otros métodos para acceder a los miembros static, cuando se usa la palabra clave import, en la parte del encabezado del programa.

La clase Main los detecta por la forma en la que se están importando los miembros, el asterisco les indica que importe todo.

Miembros static

```
import static com.anncode.operaciones.Calculadora.*
import static java.lang.Math.*;

public class Principal{

    public static void main(String[] args){
        System.out.println(suma(3, 5));
        System.out.println(PI);
    }

}
```

Nota: Cuando creamos una clase que contiene métodos, pero no atributos es incorrecto a nivel convencional llamar dichos métodos atraves de objetos, por esta razón se opta por declararlos como estáticos.

¿Y por qué?... los objetos tienen propiedades y atributos, es por eso por lo que convencionalmente es incorrecto.

Final: Variables y Contantes

Miembros final

```
public class Calculadora{
    public static final double
    PI = 3.1415926
}
```

```
Calculadora.PI;
```

final indica que la variable se convierte en constante y usualmente viene acompañado de static, dado que la mayoría de las variables constantes son usadas globalmente, sin embargo, solo depende de la lógica del programa.

Al declararlas estas se escriben con MAYUSCULAS.

Sobrecarga de Métodos y Constructores

Sobrecarga

```
public class Calculadora{
    public int suma(int a, int b){
        return a+b;
    }
    public float suma(float a, float b){
        return a+b;
    }
    public float suma(int a, float b){
        return a+b;
    }
}
```

Se conoce como sobrecarga a la acción de llamar a dos métodos o mas con el mismo nombre, pero con diferentes argumentos

Sobrecarga de Constructores

La sobrecarga de constructores se usa para **inicializar objetos**

Sobrecarga de Constructores

```
public class Doctor {
    static int id = 0; //Autoincrement
    String name;
    String speciality;

    public Doctor(){

    }

    public Doctor(String name, String speciality){
        this.name = name;
        this.speciality = speciality;
    }
}
```

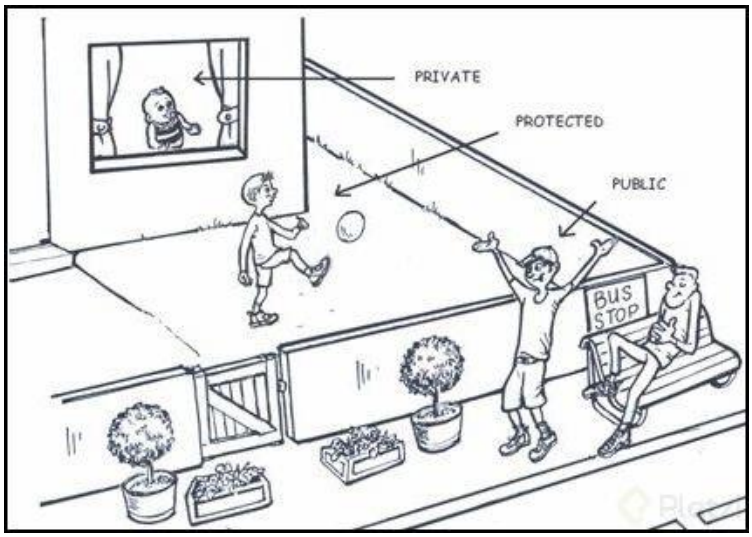
this hace referencia a todos los elementos que componen esa clase.

Encapsulamiento: Modificadores de acceso

En java al encapsulamiento también se le conoce como “hide o hidden”, o traducido como esconder, mantenerlo escondido.

Sobrecarga de Constructores

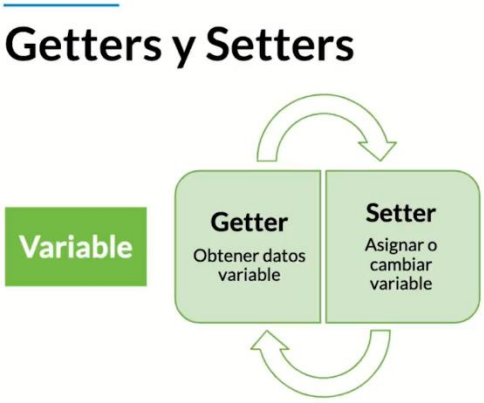
Modificador	Clase	Package	Subclase	Otros
public	✓	✓	✓	✓
protected	✓	✓	✓	●
default	✓	✓	●	●
private	✓	●	●	●



GETTERS Y SETTERS

Los Getters y Setters nos permiten leer y escribir (respectivamente) los valores de nuestras variables privadas desde fuera de la clase donde fueron creadas. Con los Getters obtenemos los datos de las variables y con los Setters asignamos o cambiamos su valor.

También puedes usar los atajos de tu IDE favorito para generar los métodos getters y setters de todas o algunas de tus variables.




```java
public class Patient {
  private String name;

  public String getName() {
    return "Patient name is " + this.name;
  }

  public void setName(String newName) {
    this.name = newName;
  }
}
```

Neatbeans

- En la clase se presiona Alt + Insert y aquí podemos agregar los getter y setters correspondientes a la clase.

Nota

- Las clases con getter and setter se conocen POJO (Plain Old Java Object)

Variables vs Objetos

Variables ≠ Objetos

Variables son entidades elementales (**muy sencillas**)

- Un número
- Un caractér
- Un valor verdadero falso

Objetos son entidades **complejas** que pueden estar formadas por la **agrupación de muchas variables y métodos.**

Clases Wrapper / Objeto primitivo

Byte
Short
Integer
Long
Float
Double
Character
Boolean

String

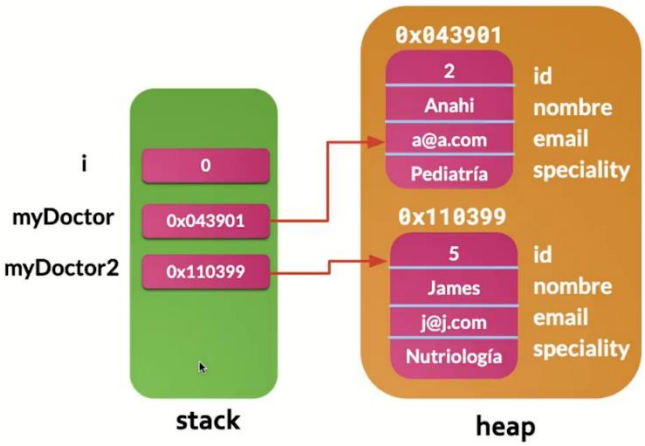
Vistazo en memoria

Un objeto es una referencia a un espacio en memoria. Cuando creamos objetos, Java los guarda en la memoria y nos devuelve coordenadas con las que podremos acceder a la información que almacenamos.

Existen dos tipos de memoria: Stack y Heap.

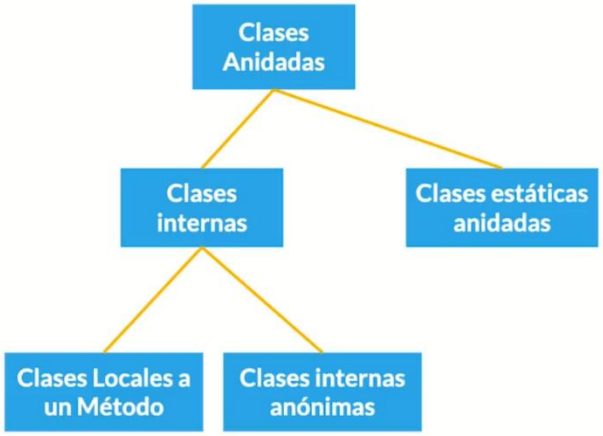
La memoria Stack es mucho más rápida y nos permite almacenar nuestra información de forma "ordenada". Aquí se guardan las variables y sus valores de tipos de datos primitivos (booleanos, números, strings, entre otros).

Los objetos también usan la memoria Stack, pero no para guardar su información, sino para guardar las coordenadas a la verdadera ubicación del objeto en la memoria Heap, una memoria que nos permite guardar grandes cantidades de información, pero con un poco menos de velocidad.



Clases Anidadas

Las Clases Anidadas o Clases Helper son clases dentro de otras clases que agrupamos por su lógica y/o características en común.



Podemos encontrar clases estáticas anidadas, clases internas que son locales a un método o clases internas anónimas. Las clases anidadas pueden llamar a cualquier tipo de elemento o método de nuestras clases.

Las Clases Estáticas no necesitan ser instanciadas para poder ser llamadas y ejecutadas, aunque debes recordar que solo permiten llamar a los métodos estáticos de sus clases padre.

```
class ClaseExterior {

  static class ClaseStaticaAnidada {

  }

  class ClaseInterna {

  }

}
```

CLASES ANIDADAS

```
public class Enclosing {
  private static int x = 1;

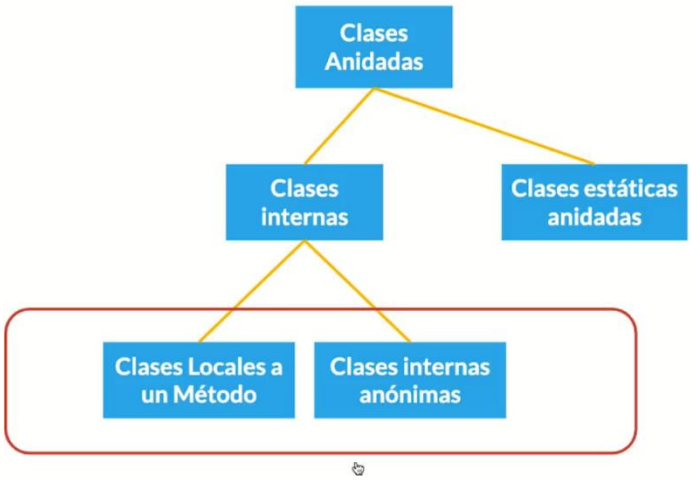
  public static class StaticNested {
     private void run() {
        //implementación
     }
  }

}


public class Main {
  public static void main(String[] args){
     Enclosing.StaticNested nested = new Enclosing.StaticNested();
     nested.run();
  }
}
```

CLASES ESTÁTICAS

Clases internas y Locales a un método



[Clases Internas en Java | Java desde Cero]

- Clases Internas

También llamadas Clases anidadas no estáticas. Al no ser una Clase estática, es necesario instanciarla. Ejemplo:

```
// Declaración de la Clase Interna
public class Outer {
    public class Inner {

    }
}
```

Llamar a la Clase:

```
public class Main {
    public static void main(String[] args) {
      Outer outer = new Outer();
      Outer.Inner inner = outer.new Inner();
    }
}
```

Para declarar este tipo de Clases hay que instanciar 2 objetos, lo que consume más espacio en memoria, por lo tanto, hay que tener cuidado al usar este tipo de Clases, de hecho este tipo de Clases no son muy comunes.

```
public class Outer {
  public class Inner {

  }

}


public class Main {
  public static void main(String[] args){
     Outer outer = new Outer();
     Outer.Inner inner = outer.new Inner();
  }
}
```

Clases Internas

- **Clases Locales a un Método**

  Este tipo de Clases tienen un scope definido para el método al que pertenecen, en el ejemplo de abajo ese método sería el primer void run().

  Si se quiere tener acceso al método run de la Clase Local, hay que crear una instancia de dicha Clase.

  Declarar una Clase Local a un método

```
public class Enclosing {
    void run() {
        class Local {
            void run() {
                ...
            }
        }
        Local local = new Local();
        local.run();
    }
}
```

Instanciar una Clase Local a un método

```
public class Main {
    public static void main(String[] args) {
          Enclosing enclosing = new Enclosing();
        enclosing.run();
    }
}
```

Este tipo de Clases también consumen más espacio en memoria. Lo recomendable en estos casos es usar Clases estáticas anidadas para hacer un mejor uso de la memoria.

```
public class Enclosing {
  void run() {
     class Local {
        void run() {

        }
     }

     Local local = new Local();
     local.run();
  }

}

public class Main {
  public static void main(String[] args){
     Enclosing enclosing = new Enclosing();
     enclosing.run();
  }
}
```

Clases Locales a un Método

Enumerations

Los enumerations son tipos de datos muy especiales pues este, es el único en su tipo que sirve para declarar una colección de constantes, al ser así estaremos obligados a escribirlos con mayúsculas.

Usaremos enum cada vez que necesitemos representar un conjunto fijo de constantes. *Por ejemplo, los días de la semana*.

Así podemos declarar un enumeration usando la palabra reservada **enum.**

```
public enum Day {
        SUNDAY, MONDAY, TUESDAY, WEDNESDAY,
        THURSDAY, FRIDAY, SATURDAY
}
```

Puedo crear referencias de enumerations de la siguiente forma:

```
Day day;
switch (day) {
        case MONDAY:
                System.out.println("Mondays are good.");
                break;
        case FRIDAY:
                System.out.println("Fridays are nice");
                break;
        case SATURDAY: case: SUNDAY:
                System.out.println("Weekends are the best");
                break;
        default:
                System.out.println("Midweek are so-so");
                break;

}
```

Y puedo llamar un valor del enumeration así:

```
Day.MONDAY;
Day.FRIDAY;
Day.SATURDAY
```

Los enumerations pueden tener atributos, métodos y constructores, como se muestra:

```
public enum Day {
  MONDAY("Lunes");
  TUESDAY("Jueves");
  FRIDAY("Viernes");
  SATURDAY("Sábado");
  SUNDAY("Domingo");

  private String spanish;
  private Day(String s) {
    spanish = s;
  }

  private String getSpanish() {
    return spanish;
  }
}
```

Y para utilizarlo lo podemos hacer así:

```
System.out.println(Day.MONDAY);
```

Imprimirá: **MONDAY**

```
System.out.println(Day.MONDAY.getSpanish());
```

Imprimirá: **Lunes**

Datos curiosos sobre los **enum:**

1. Puedes crear **enums** dentro de clases, no solo como un archivo aparte:

Main.java | *Doctor.java | UIMenu.java | Patient.java

```java
import java.util.ArrayList;

public class Doctor {
//  State/Attributes
    public static int id = 0;
    public String name;
    public String specialty;

//  Enumerations
    public enum Day{
        SUNDAY("Domingo", "Dimanche"),
        MONDAY("Lunes", "Lundi"),
        TUESDAY("Martes", "Mardi"),
        WEDNESDAY("Miércoles", "Mercredi"),
        THURSDAY("Jueves", "Jeudi"),
        FRIDAY("Viernes", "Vendredi"),
        SATURDAY("Sábado", "Samedi");

        private String spanish;
        private String french;

        private Day(String s, String f) {
            spanish = s;
            french = f;
        }

        public String getSpanish() {
            return spanish;
        }

        public String getFrench() {
            return french;
        }
    }
```

2. Los **enums** son en realidad constantes estáticas
   Este código:

```
//  Enumerations
    public enum Day{
        SUNDAY,
        MONDAY
```

   Es lo mismo que este código:

```
private static final String SUNDAY = "SUNDAY";
private static final String MONDAY = "MONDAY";
```

3. Para acceder a os atributos de los **enum** hacemos lo siguiente (como hizo Anahí con el método getSpanish()):
   a. Agregamos dentro de paréntesis el valor del atributo que deseemos, por ejemplo, añadiremos las palabras *Domingo* y *Lunes* a sus respectivos **enums** en inglés:

```
//  Enumerations
    public enum Day{

        SUNDAY("Domingo"),
        MONDAY("Lunes");
```

   b. En seguida necesitaremos agregar un constructor privado con un parámetro. Puedes nombrarlo como gustes, lo importante es que lleve el mismo tipo del atributo, en este caso String:

```
private Day(String s) {

}
```

   NOTA: Puedes tener atributos de cualquier tipo, primitivos u objetos.
   c. La variable *s* es la que contiene el valor de nuestro primer atributo, así que para recuperarla y utilizarla hacemos los siguiente:

```
private String spanish;

private Day(String s) {
    spanish = s;
}

public String getSpanish() {
    return spanish;
}
```

1. Agregamos una variable privada String llamada *spanish* (o como gusten llamarla una vez más, español, traduccion_al_español, etc).
2. Asignamos nuestra variable nueva a la variable *s* dentro del constructor.
3. Agregamos un getter que nos permita recuperar el valor de nuestra variable *spanish* con el valor recuperado de la variable *s*.

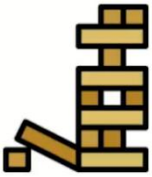
Listo, el cómo mandar a llamar nuestro nuevo método ya lo explicó ***anncode***, sólo quería dejar en claro algunos puntos que pudieron haberse explicado de mejor forma y con más profundidad, atando todo lo que hemos venido aprendiendo. El punto 3 es el que más me costó entender, espero no haber dejado más perdido a alguien con mi explicación.

Reutilizar código

¿Qué es la herencia?... DRY Don´t Repeat Yourself

DRY: *Don't repeat yourself*

- Promueve la reducción de duplicación en programación
- Las piezas de información **nunca deben duplicarse.**
- **Incrementa la dificultad** en los cambios y evolución



La Herencia consiste en crear nuevas clases a partir de otras clases, establecemos una relación padre e hijo entre nuestras clases. Es diferente a las clases anidadas, ya que, en vez de crear clases dentro de clases, le indicamos a nuestras subclases de qué superclase pueden heredar (extends) para reutilizar el código de algunos de sus métodos.

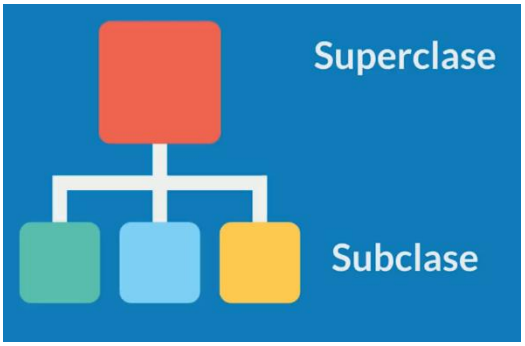
Recuerda que nuestras clases no pueden heredar de más de una clase.



Herencia

Se establece una relación **padre e hijo**





Super y This

Super indica que una variable o método es de la clase padre, la superclase de cual heredan nuestras subclases, solo la usamos cuando aplicamos herencia.

Además, podemos llamar al constructor de la clase padre desde sus diferentes subclases usando super(); y enviando los argumentos que sean necesarios.

Por otro lado, this nos permite especificar que nuestras variables están señalando a la misma clase donde estamos trabajando, ya sea una clase normal, anidada, subclase o superclase.

Herencia

```
public class User {

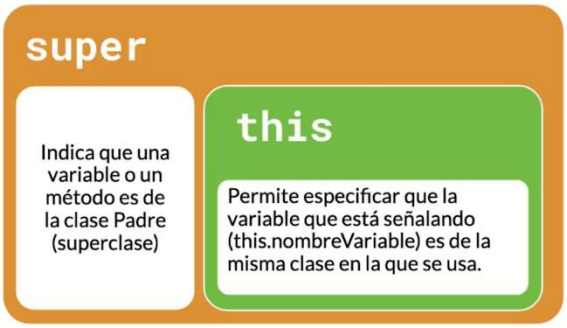
}
public class Patient extends User {

}
```

Súper Clase

Subclase

super y this



Tarea

Investiga la clase object y para qué sirve el método toString()

Polimorfismo: Sobreescritura de Métodos

Polimorfismo

Posibilidad de sobreescribir un método con **comportamientos diferentes**.

El Polimorfismo es una característica de la programación orientada a objetos que consiste en sobrescribir algunos métodos de la clase de la cual heredan nuestras subclases para asignar comportamientos diferentes.

Además de los métodos de las superclases, también podemos redefinir el comportamiento de los métodos que "heredan" todos nuestros objetos, así como .toString, hashCode, finalize, notify, entre otros.

Sobreescritura de Constructores

Un constructor en una subclase usando los miembros heredados de la superclase **con argumentos diferentes.**

La sobreescritura de constructores consiste en usar los miembros heredados de una superclase pero con argumentos diferentes.

Recuerda que no podemos sobrescribir los métodos marcados como final o static.

Sobreescritura

Cuando una clase hereda de otra y en esta **clase hija se redefine un método** con una implementación distinta a la de la clase padre

Interfaces

Interfaces

Es un tipo de referencia similar a una clase que podría contener solo **constantes** y **definiciones de métodos**.

Las **Interfaces** son un tipo de referencia similar a una clase con solo constantes y definiciones de métodos, son de gran ayuda para definir los comportamientos que son redundantes y queremos reutilizar más de una clase, incluso cuando tenemos muchas clases y no todas pertenecen a la misma "familia".

Las interfaces establecen la forma de las clases que la implementan, así como sus nombres de métodos, listas de argumentos y listas de retorno, pero NO sus bloques de código, eso es responsabilidad de cada clase.

Interfaces

Se establece la forma de una clase (nombres de métodos, listas de argumentos y tipos de retorno, pero **no bloques de código**).

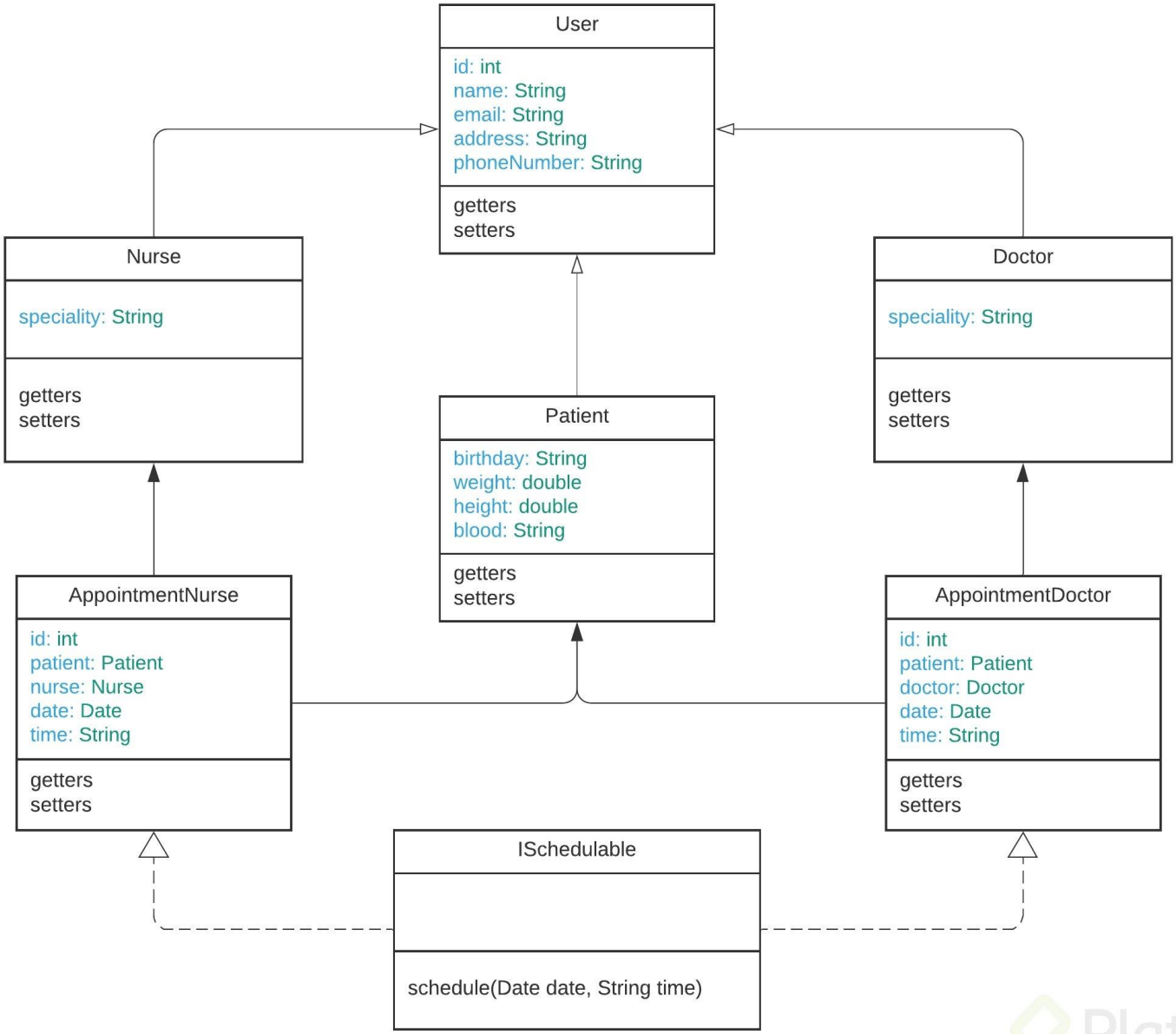
Composición de Interfaces en Clases

Abstraer todos los métodos/comportamientos de una clase para modularizarlos (comprimirlos, encapsularlos) en una interfaz y reutilizar su código en diferentes clases.

Las interfaces se crean utilizando la palabra reservada interface y se implementan en nuestras clases con implements.

Recuerda que podemos heredar (implementar) más de una interfaz, pero no podemos hacerlo de las clases padres o superclases.

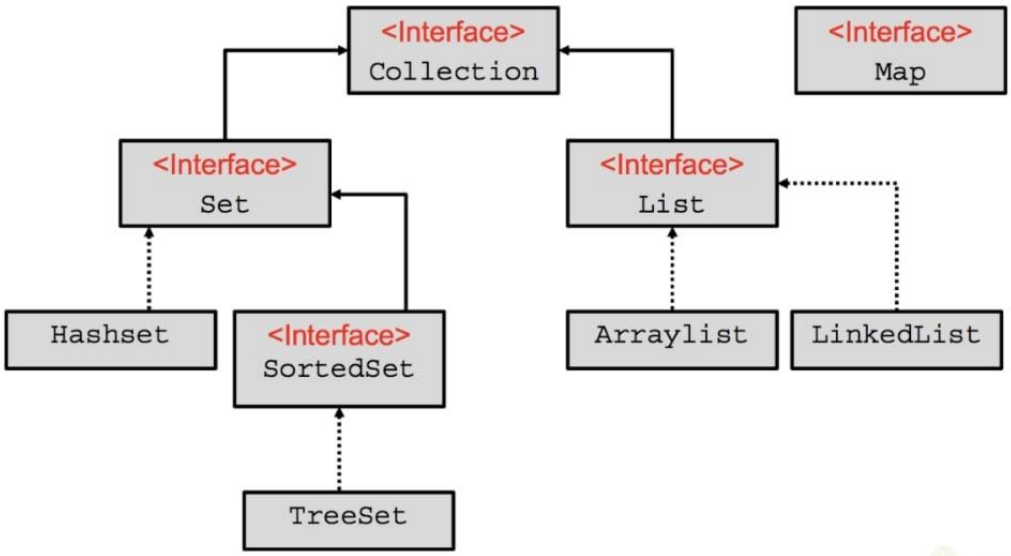
Así se va siendo la estructura UML de nuestro proyecto…



Platzi

Collections

Otras interfaces que son muy importantes en Java son los llamados **Collections**
Los Collections nos van a servir para trabajar con colecciones de datos, específica y solamente con objetos, para esto recuerda que tenemos disponibles nuestras clases Wrapper que nos ayudan a convertir datos primitivos a objetos.
Los collections se diferencian de los arrays en que su tamaño no es fijo y por el contrario es dinámico.
A continuación, te muestro un diagrama de su composición:



Platzi

Como podemos observar el elemento más alto es la interfaz **Collection**, para lo cual, partiendo de su naturalidad de interface, entendemos que tiene una serie de métodos "básicos" dónde su comportamiento será definido a medida que se vaya implementando en más elementos. De ella se desprenden principalmente las interfaces **Set** y **List.**

La interface **Set** tendrá las siguientes características:

Almacena objetos únicos, no repetidos.
La mayoría de las veces los objetos se almacenarán en desorden.
No tenemos índice.

La interface **List** tiene estas características:

Puede almacenar objetos repetidos.
Los objetos se almacenan en orden secuencial.
Tenemos acceso al índice.

- Si seguimos analizando las familias tenemos que de **Set** se desprenden:

Clase HashSet
Interfaz SortedSet y de ella la clase TreeSet.

HashSet los elementos se guardan en **desorden** y gracias al mecanismo llamado hashing (obtiene un identificador del objeto) **permite almacenar objetos únicos.**

TreeSet almacena **objetos únicos**, y gracias a su estructura de árbol el **acceso* es sumamente **rápido.**

- Ahora si analizamos la familia **List**, de ella se desprenden:

Clase **ArrayList** puede tener duplicados, no está sincronizada por lo tanto es más rápida
Clase **Vector** es sincronizada, los datos están más seguros, pero es más lento.
Clase **LinkedList**, puede contener elementos duplicados, no está sincronizada (es más rápida) al ser una estructura de datos doblemente ligada podemos añadir datos por encima de la pila o por debajo.

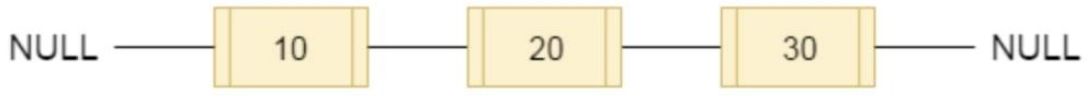


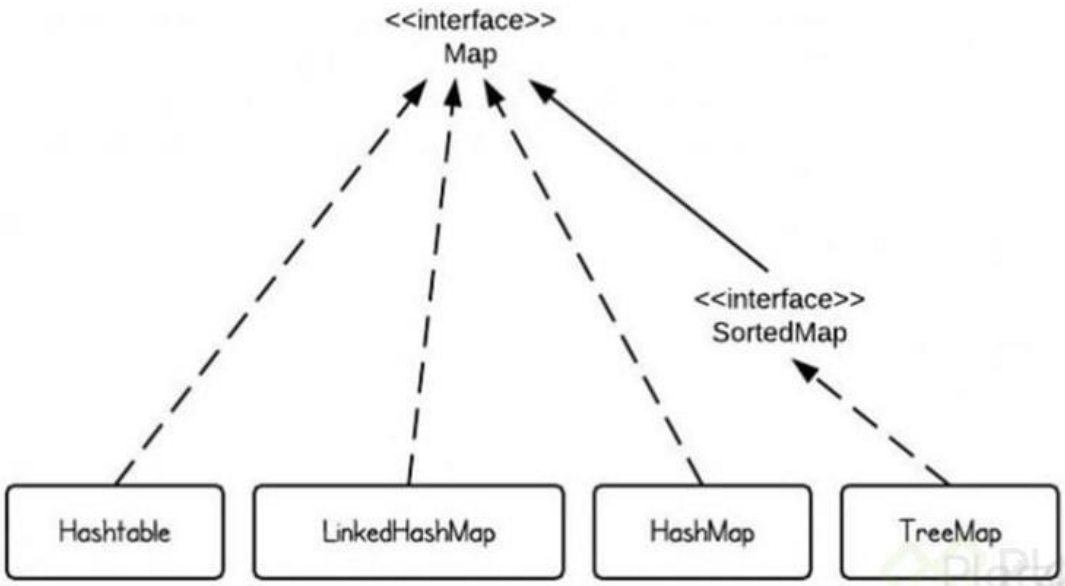
fig- doubly linked list

Platzi

Sigamos con Map

Lo primero que debes saber es que tiene tres implementaciones:

HashTable
LinkedHashMap
HashMap
SortedMap ➡️ TreeMap



La interfaz **Map** no hereda de la interfaz Collection porque representa una estructura de datos de Mapeo y no de colección simple de objetos. Esta estructura es más compleja, pues cada elemento deberá venir en pareja con otro dato que funcionará como la llave del elemento.

Map

Donde K es el key o clave
Donde V es el value o valor

Podemos declarar un map de la siguiente forma:

```
Map<Integer, String> map = new HashMap<Integer, String>();
Map<Integer, String> treeMap = new TreeMap<Integer, String>();
Map<Integer, String> linkedHashMap = new LinkedHashMap<Integer, String>();
```

Como observas solo se puede construir el objeto con tres elementos que implementan de ella: **HashMap**, **TreeMap** y **LinkedHashMap** dejando fuera HashTable y SortedMap. SortedMap estará fuera pues es una interfaz y HashTable ha quedado deprecada pues tiene métodos redundantes en otras clases. Mira la funcionalidad de cada uno.

Como te conté hace un momento Map tiene implementaciones:

HashMap: Los elementos no se ordenan. No aceptan claves duplicadas ni valores nulos.
LinkedHashMap: Ordena los elementos conforme se van insertando; provocando que las búsquedas sean más lentas que las demás clases.
TreeMap: El Mapa lo ordena de forma "natural". Por ejemplo, si la clave son valores enteros (como luego veremos), los ordena de menos a mayor.

Para iterar alguno de estos será necesario utilizar la interface **Iterator** y para recorrerlo lo haremos un bucle while así como se muestra:

- Para HashMap

```
// Imprimimos el Map con un Iterador
Iterator it = map.keySet().iterator();
while(it.hasNext()){
  Integer key = it.next();
  System.out.println("Clave: " + key + " -> Valor: " + map.get(key));
}
```

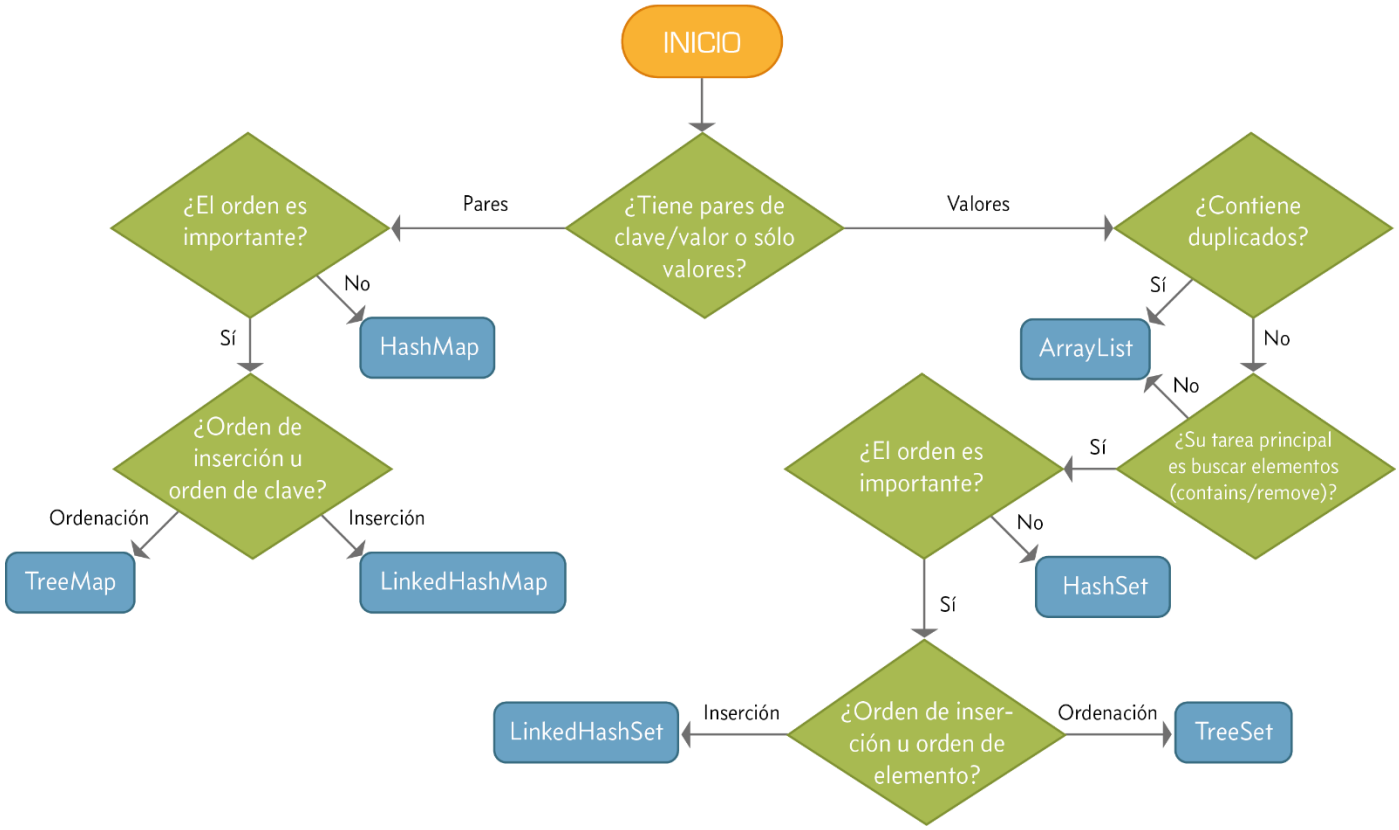
- Para LinkedHashMap

```
// Imprimimos el Map con un Iterador
Iterator it = linkedHashMap.keySet().iterator();
while(it.hasNext()){
  Integer key = it.next();
  System.out.println("Clave: " + key + " -> Valor: " + linkedHashMap.get(key));
}
```

- Para TreeMap

```
// Imprimimos el Map con un Iterador
Iterator it = treeMap.keySet().iterator();
while(it.hasNext()){
  Integer key = it.next();
  System.out.println("Clave: " + key + " -> Valor: " + treeMap.get(key));
}
```

Diagrama de decisión para uso de colecciones Java



Aplicar abstracción

Clases abstractas

A veces NO necesitamos implementar todos los métodos de una clase heredada o interfaz. No siempre necesitamos crear instancias o implementar todos los métodos heredados de una clase padre, así como tampoco podremos necesitamos algún método de nuestras interfaces, pero estas nos obligan a escribir el código de todos los métodos que definimos genéricamente.

Afortunadamente, las **Clases Abstractas** resuelven todos estos problemas. Son una combinación entre interfaces y herencia donde no implementaremos todos los métodos ni tampoco crearemos instancias.
Una restricción de las clases abstractas es que no podemos crear objetos ni instancias.

```
public abstract class Figura {
  // ...
}
```

```
class Triangulo extends Figura {
  // ...
}
```

Polimorfismo

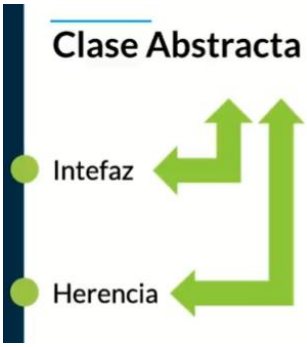
Herencia Clases
Métodos sobreescritos
Muchas formas
Implementación **Interfaces**

Interfaces

A veces no necesitamos implementar todos los métodos

Herencia

Las clases podrían no necesitar heredar la implementación de un método
A veces **no necesitamos crear instancias** de una clase padre, ya que es muy genérica



Clase Abstracta

No implementaremos todos los métodos

No crearemos instancias

Miembros Abstractos

Los Métodos Abstractos son los métodos que debemos implementar obligatoriamente cada vez que usemos nuestras clases abstractas, mientras que los métodos que no sean abstractos van a ser opcionales.

```
public abstract class Figura {
  abstract void dibujar(); // obligatorio
  void dibujar3D(); // no es obligatorio
}

class Triangulo extends Figura {
  void dibujar() {
    // Instrucciones para dibujar el triángulo...
  }
}
```

Recuerda los métodos abstractos solo se pueden implementar en clases abstractas. Y las clases abstractas no necesitan ser instanciadas para ser implementadas.
La diferencia entre una clase abstracta y una interface, es que al crear un método abstracto en la clase abstracta (super_clase), se obliga heredar este método a las clases hijas (sub_clases), en cambio al implementar una interface en una clase ya sea abstracta o concreta, debemos implementar TODOS los métodos que tenga dicha interface.

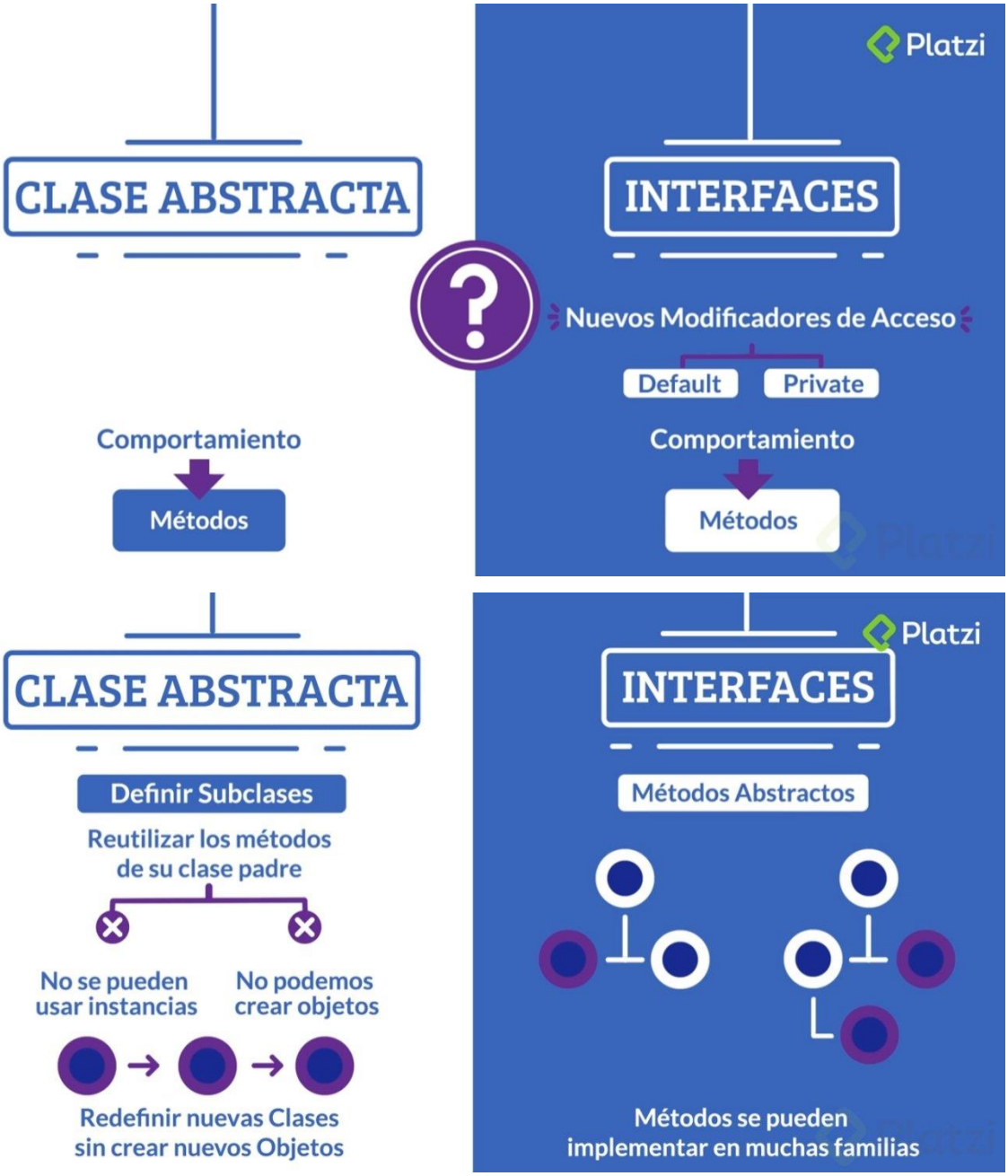
Clases anónimas

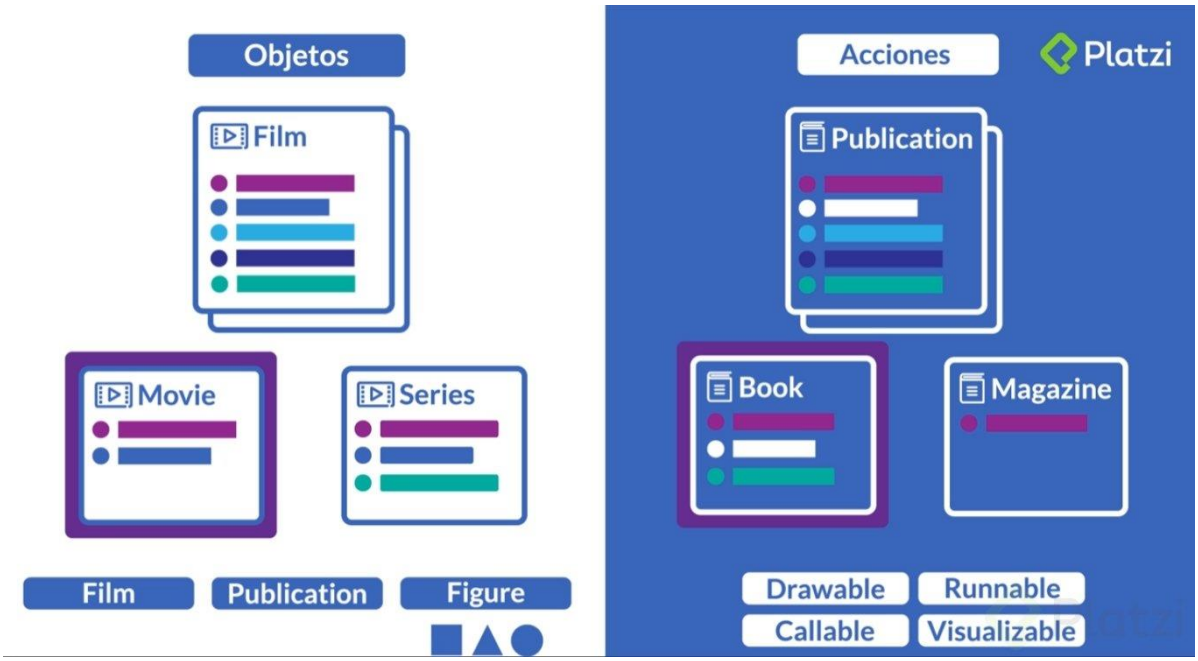
Las Clases Anónimas son una forma de instanciar clases abstractas sin necesidad de usar sus clases hijas. Pero este tipo de instanciación tiene algunas restricciones: el ciclo de vida de estas instancias NO es duradero, no las tendremos disponibles durante toda la ejecución del programa.

```
// Clase Abstracta:
public abstract class Figura {
  abstract void dibujar();
}

// Clase Anónima:
User user = new User() {
  @Override
  public void showDataUser() {
    // Instrucciones...
  }
};
```

Diferencias entre las Interfaces vs Clases Abstractas




Objetos
Film
Movie
Series
Film
Publication
Figure
Acciones
Platzi
Publication
Book
Magazine
Drawable
Runnable
Callable
Visualizable


Platzi
ABSTRACTO
Interfaces
Implementación
Crear buenas abstracciones
Comportamiento Común
Declaración de Métodos

Interfaces en Java 8 y 9

Un método que no tenga comportamientos ya se considera abstracto aunque no tenga la palabra reservada abstract

```java
public interface MyInterface {
    default void defaultMethod() {
        privateMethod("Hello from the default method!");
    }
    private void privateMethod(final String string) {
        System.out.println(string);
    }
    void normalMethod();
}
```

default y private Methods

Modificador	Clase	Package	Subclase	Otros
public	✓	✓	✓	✓
protected	✓	✓	✓	●
default	✓	✓	●	●
private	✓	●	●	●

Herencia en interfaces

Las interfaces pueden heredar de otras interfaces utilizando la palabra clave extends, el concepto de herencia se aplicará como naturalmente se practica en clases, es decir, la interfaz heredará y adquirirá los métodos de la interfaz padre.

Una cosa interesante que sucede en caso de herencia con interfaces es que, aquí sí es permitido la herencia múltiple como ves a continuación:

```
public interface IReadable {
  public void read();
}


public interface Visualizable extends IReadable, Serializable {
  public void setViewed();
  public Boolean isViewed();
  public String timeViewed();
}
```

Además, siguiendo las implementaciones de métodos default y private de las versiones Java 8 y 9 respectivamente podemos sobrescribir métodos y añadirles comportamiento, si es el caso.

```
public interface Visualizable extends IReadable, Serializable {
  public void setViewed();
  public Boolean isViewed();
  public String timeViewed();

@Override
  default void read() {
  // TODO Auto-generated method stub
}
}
```

Tareas

Estudia excepciones

Estudia métodos String

Estructuras de árbol

Arrylist, map, tremap